

Chapter 13: Tweepy Functions

Understanding and Implementing Tweepy Functions
for Twitter API

Write a Python code to fetch user information from Twitter's API using the requests package and a Bearer Token.

- `import requests`
- `# Replace with your actual Bearer Token from X Developer Portal`
- `BEARER_TOKEN = "GIVE YOUR BEARER TOKEN"`
- `def get_user_info(username):`
- `url =`
 `f"https://api.twitter.com/2/users/by/username/{username}?user.fields=id,name,username,description,public_metrics"`
 `"`
- `headers = {`
- `"Authorization": f"Bearer {BEARER_TOKEN}"`
- `}`
- `response = requests.get(url, headers=headers)`
- `if response.status_code == 200:`
- `user_data = response.json()["data"]`
- `print("User ID:", user_data["id"])`
- `print("User Name:", user_data["name"])`
- `print("Screen Name:", user_data["username"])`
- `print("Description:", user_data["description"])`
- `print("Followers Count:", user_data["public_metrics"]["followers_count"])`
- `else:`
- `print("Error:", response.status_code, response.text)`
- `# Replace 'elonmusk' with any Twitter username`
- `get_user_info("GIVE YOUR TWITTER USERNAME")`

Write a Python code to fetch user details, including follower count, from Twitter's API using the tweepy package and authentication credentials.

- `import tweepy`
- `# Twitter API credentials`
- `API_KEY = " GIVE YOUR API KEY"`
- `API_SECRET = " GIVE YOUR API SECRET"`
- `ACCESS_TOKEN = " GIVE YOUR ACCESS TOKEN"`
- `ACCESS_SECRET = "GIVE YOUR ACCESS SECRET"`
- `BEARER_TOKEN = "GIVE YOUR BEARAER TOKEN"`
- `# Authenticate with Twitter API v2`
- `client = tweepy.Client(bearer_token=BEARER_TOKEN)`
- `# Twitter username (replace with your username)`
- `username = "GIVE YOUR TWITTER USERNAME"`
- `# Fetch user details`
- `user = client.get_user(username=username, user_fields=["public_metrics"])`
- `# Get follower count`
- `if user.data:`
- `print("User ID:", user.data.id)`
- `print("Username:", user.data.username)`
- `print("Name:", user.data.name)`
- `print("Followers Count:", user.data.public_metrics["followers_count"])`
- `else:`
- `print("User not found.")`

Write a Python code to search for recent tweets containing a specific keyword using the tweepy package and Twitter API authentication.

- `import tweepy`
- `API_KEY = " GIVE YOUR API KEY"`
- `API_SECRET = " GIVE YOUR API SECRET"`
- `ACCESS_TOKEN = " GIVE YOUR ACCESS TOKEN"`
- `ACCESS_SECRET = "GIVE YOUR ACCESS SECRET"`
- `client = tweepy.Client(bearer_token="GIVE YOUR BEARAER TOKEN")`
- `query = "Python programming" # Replace with your search term`
- `tweets = client.search_recent_tweets(query=query, max_results=15)`
- `# Print tweets`
- `for tweet in tweets.data:`
- `print(f"Tweet: {tweet.text}\n")`

tweepy.OAuthHandler

- Used to create an OAuthHandler object for authentication.
- Example:
- `import tweepy`
- `auth = tweepy.OAuthHandler('API_KEY', 'API_SECRET')`

OAuthHandler.set_access_token

- Used to set the access token and secret.
- Example:
- `auth.set_access_token('ACCESS_TOKEN', 'ACCESS_TOKEN_SECRET')`

api=tweepy.API

- Used to create an API object for interacting with Twitter.
- Example:
- `api = tweepy.API(auth)`

api.get_user()

- Used to get information about a Twitter account.
- Example:
- `user = api.get_user(screen_name='TwitterUser')`
- `print(user.name, user.followers_count)`

api.followers

- Used to get a list of followers for an account.
- Example:
- `followers = api.followers(screen_name='TwitterUser')`
- `for f in followers:`
- `print(f.name)`

api.followers_count

- Returns the number of followers.
- Example:
- `user = api.get_user(screen_name='TwitterUser')`
- `print(user.followers_count)`

api.followers_ids

- Used to get a list of follower IDs for an account.
- Example:
- `follower_ids =
api.followers_ids(screen_name='TwitterUser')`
- `print(follower_ids)`

api.friends

- Used to get a list of friends (accounts followed) for an account.
- Example:
- `friends = api.friends(screen_name='TwitterUser')`
- `for f in friends:`
- `print(f.name)`

api.friends_count

- Used to get number of friends (accounts followed).
- Example:
- `user = api.get_user(screen_name='TwitterUser')`
- `print(user.friends_count)`

api.me()

- Returns a User object for the authenticated account.
- Example:
- `me = api.me()`
- `print(me.name, me.screen_name)`

Tweepy.cursor

- Handles pagination and rate limits when accessing large datasets.
- Example:
- `for tweet in tweepy.Cursor(api.user_timeline, screen_name='TwitterUser').items(10):`
- `print(tweet.text)`

api.user_timeline

- Used to get tweets from a user's timeline.
- Example:
- ```
tweets =
api.user_timeline(screen_name='TwitterUser',
count=5)
```
- ```
for tweet in tweets:
```
- ```
 print(tweet.text)
```



# api.home\_timeline

- Used to get tweets from the home timeline.
- Example:
- `tweets = api.home_timeline(count=5)`
- `for tweet in tweets:`
- `print(tweet.text)`

# api.search

- Used to search for tweets matching a query.
- Example:
- `tweets = api.search(q='Python', count=5)`
- `for tweet in tweets:`
- `print(tweet.text)`



# api.trends\_available

- Gets a list of locations with trending topics.
- Example:
- `trends = api.trends_available()`
- `print(trends)`

# api.trends\_closest

- Finds locations close to a specified latitude and longitude.
- Example:
- `closest_trends = api.trends_closest(lat=37.7749, long=-122.4194)`
- `print(closest_trends)`



# api.trends\_place

- Gets trending topics for a specific location.
- Example:
- `trends = api.trends_place(1)`
- `print(trends)`

# WordCloud

- Used to create a word frequency cloud.
- Example:
- `from wordcloud import WordCloud`
- `wordcloud = WordCloud().generate('sample text')`
- `wordcloud.to_image().show()`



# clean(text)

- Cleans a tweet by removing URLs, mentions, hashtags, emojis, reserved words, and numbers.
- Example:
- `from preprocessor import clean`
- `tweet = 'Check this out! http://example.com #AI @user'`
- `print(clean(tweet))`



# tokenize(text)

- Tokenizes the tweet while keeping URLs, mentions, and hashtags.
- Example:
- `from preprocessor import tokenize`
- `tweet = 'Hello #World! Visit http://example.com'`
- `print(tokenize(tweet))`



# set\_options(\*options)

- Specifies what to remove (e.g., mentions, URLs, hashtags).
- Example:
- `from preprocessor import set_options, clean`
- `set_options('urls', 'hashtags')`
- `tweet = 'Visit http://example.com #Python'`
- `print(clean(tweet))`



# clear()

- Clears the set options, resetting the preprocessor.
- Example:
- `from preprocessor import clear`
- `clear()`



# on\_status

- Called when a new tweet arrives.
- Example:
- `class MyListener(tweepy.StreamListener):`
- `def on_status(self, status):`
- `print(status.text)`

# on\_connect

- Called when connection to the stream is successful.
- Example:
- `def on_connect(self):`
- `print('Connected to stream.')`



# on\_limit

- Called when tweet stream exceeds rate limits.
- Example:
- `def on_limit(self, track):`
- `print('Rate limit exceeded')`

# on\_error

- Called when Twitter sends an error code.
- Example:
- `def on_error(self, status_code):`
- `print(f'Error: {status_code}')`



# on\_delete

- Called when a tweet is deleted.
- Example:
- `def on_delete(self, status_id, user_id):`
- `print(f'Tweet {status_id} deleted')`

# on\_data

- Called when raw data is received from Twitter.
- Example:
- `def on_data(self, raw_data):`
- `print(raw_data)`



# on\_timeout

- Called if the connection to the stream times out.
- Example:
- `def on_timeout(self):`
- `print('Stream timeout')`

# on\_warning

- Called when Twitter sends a disconnect warning.
- Example:
- `def on_warning(self, notice):`
- `print('Warning:', notice)`